Chapter #12:-

Physical Design

⇒ Physical design is the final phase. it is where actually implements the decisions like taking the blueprint and building the actual structure.

Why it's important?

⇒ This is not fixed, as the data mart is used, need to make changes to optimize its performance and meet new user needs.

The best approache

⇒ Logical design (Planning the structure) and physical design (building it) work best together.

Challenges:

⇒ Some tasks like creating indexes or materializing views, can improve performance but are complex.

DBMS dependencies:

⇒ The Specific DBMS use has a big impact on the physical design choices.

⇒ Different dbms has different features and capabilities.

Optimizers

What they are?

⇒ are Crucial part of Physical database design.

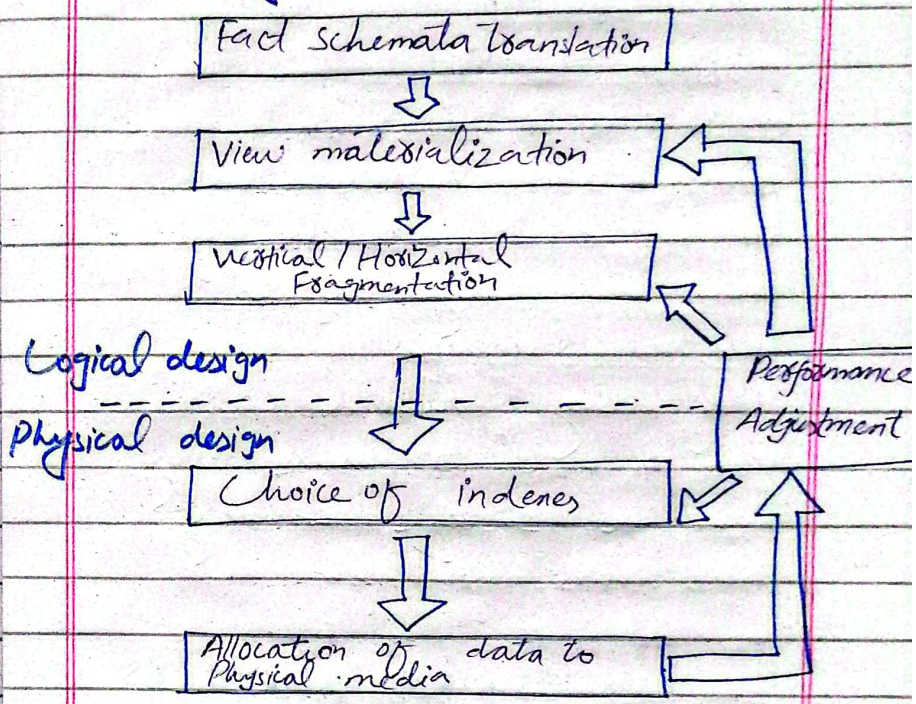
They are like smart planners that figure out the most efficient way to execute a query.

How they work?

⇒ when a query run, the optimizer analyzes it and creates a query execution Plan. This plan is step-by-step guide for database to follow. Specifying which tables to access,

⇒ what indexes to use, and the orders of operations.

RIs between Physical & Logical design:



importance of in-depth knowledge of DBMS for effective DWH:

Internal expertise:

⇒ Companies should aim to build internal expertise in DBMSs as it's a valuable asset for data warehousing projects.

External consultants:

⇒ If using external consultants, it is essential to ensure they have the specific knowledge & skills needed for DWH.

Types of Optimizers

1. **Rule-Based optimizers:**

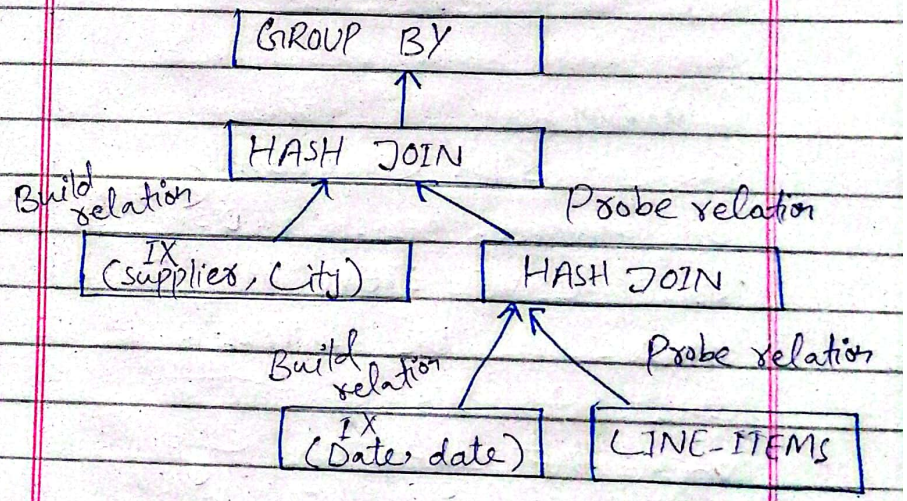
→ These optimizers rely on a set of rules to determine how to execute a query.

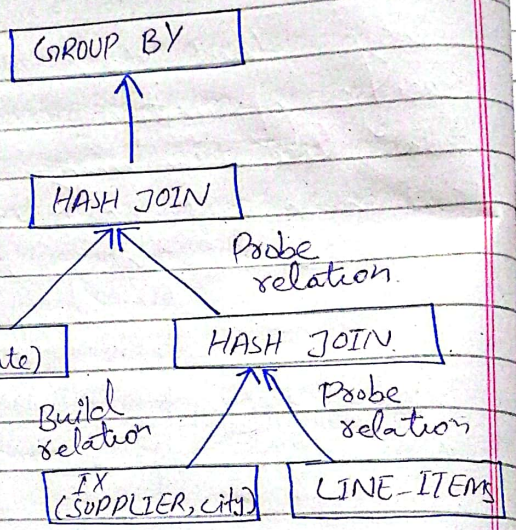
→ These rules consider the DB structure, query structure & available indexes.

Limitation:

⇒ They don't take into account specific information about the data, like how many rows are in a table or how data is distributed. This can lead to suboptimal query execution plans.

Two different ways to execute a query:



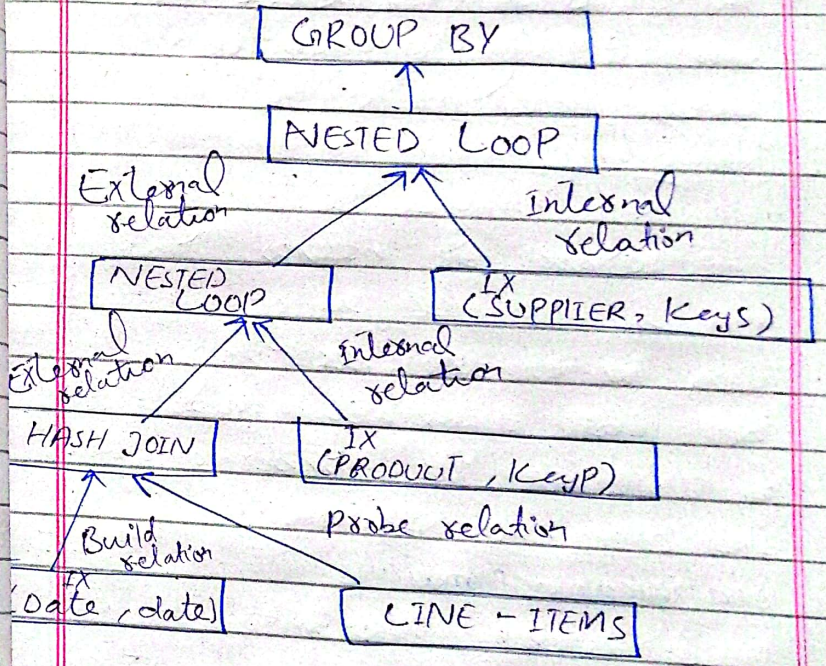


Query:

⇒ SELECT SUM (FT.quantity, DT1.Product, DT2.state, DT3.date)
FROM LINE_ITEM AS FT, PRODUCT AS DT1, SUPPLIER as DT2, DATE As DT3
WHERE FT.keyP = DT1.keyP AND
FT.keyS = DT2.keyS AND
FT.keyD = DT3.keyD AND
DT3.date = '1/1/2008'
GROUP BY DT1.Product, DT2.state, DT3.date

Example:

Execution Plan



2. Cost-Based Optimizers:

Def:

⇒ Cost-based Optimizers uses a statistical information about the data to choose the most efficient way to execute a query.

How they works?

⇒ They analyze the query and the available indexes, and then estimate the cost of different ways to execute the query. The cost is usually measured in terms of the number of disk I/O operations required.

Statistical information:

⇒ Statistical information could be includes the number of rows in each table, the number of distinct values for each attribute & the distribution of values for each attribute.

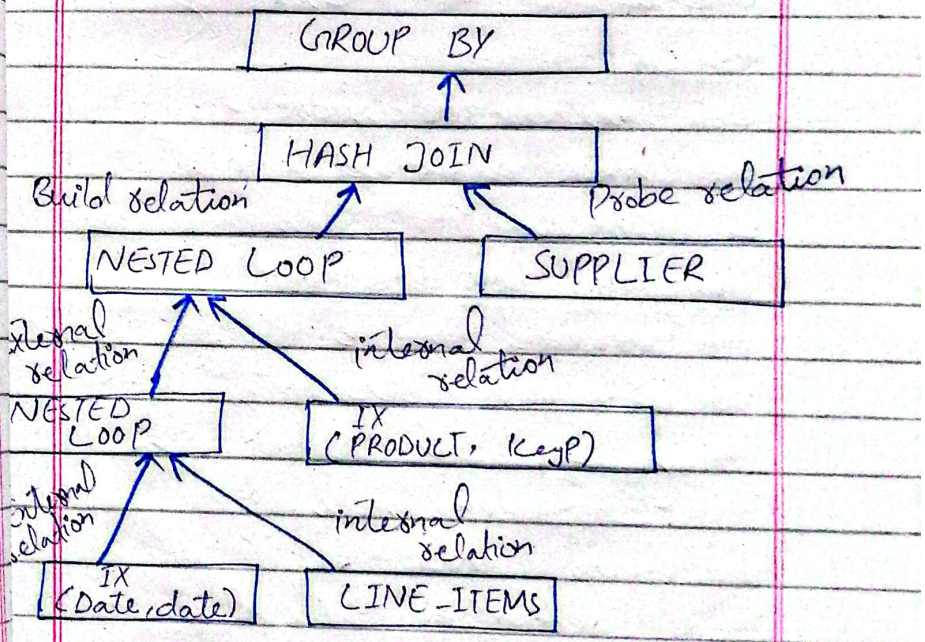
Pros:

- More accurate
- Improved Performance.

Cons:

- Dependency on statistical information
- Complexity

Diagram with example:



3. **Histograms:**

⇒ are a ways to summarize the distribution of values in a table. They divide the range of values into intervals and count the number of values that fall into each interval.

Types:

- Equi-width
- Equi-Height

Accuracy:

⇒ Equi-height are more accurate.

⇒ But more difficult to calculate & maintain

Histogram-based estimation:

⇒ Histograms can be used to estimate the selectivity of query by calculate the sum of the frequencies of the intervals that fall within the range of the condition.

Index Selection

What is it?

⇒ Index selection is the process of choosing which indexes to create on data most tables to improve query performance.

⇒ Indexes are like shortcuts that help the DB find specific data more quickly.

Challenges:

⇒ Choosing the right indexes can be difficult because there are many different types of indexes, and each index has its own advantages and disadvantages.

⇒ Creating too many indexes can slow down data updates and increase storage costs.

Two-Phase approach to index selection:

⇒ Identify useful indexes.
⇒ Select the best subset

Factors to consider when selecting indexes:
- Data Distribution
- Query Patterns
- Space constraints

★ Indexing Dimension Tables:

Purpose:
- Dimension tables are used to filter the data in fact tables.
- Indexing dimension tables can improve the performance of queries that involve filtering on dimension attributes.

Types of indexes:
⇒ Two types of indexes that can be used for DTs.
1. RID-list indexes
2. Bitmap indexes.

Choosing the right Type of index:
⇒ Depends on the following factors:
- Cardinality
- Query Selectivity
- DBMS features

★ Indexing Fact Tables:

Purpose:
⇒ Fact tables store detailed data and they are often very large.
⇒ Indexing fact tables can improve performance of query.

Primary indexes:
⇒ Fact tables typically have a primary key, which is a combination of foreign keys from dimensio tables.
⇒ However, primary indexes are not useful for primary indexes.

Join indexes:

⇒ To improve join performance it is often necessary to create additional indexes on fact tables.

⇒ These indexes are called join indexes, and they are typically created on pairs of columns that are frequently joined together.

Additional Physical Design Elements:

⇒ The Physical design of data mart does not end with defining its indexes.

⇒ Also need to define the set of parameters that determine the Physical schemata of data mart.

⇒ The disk space used up by a database is normally divided into:

- Tablespaces
- Data files
- Data blocks

Tablespaces:

- A tablespace is a logical container for database objects such as tables and indexes.
- it is a way to organize and manage disk space.

Data files:

- Also called segment
- A data file belong to only one tablespace.
- More than on data files can be associated with one tablespace.
- These are Physical files that store the actual data within a tablespace.

Data Blocks:

- These are the smallest units of data that can be read or written to disk.

Splitting a DB into Tablespaces

⇒ Although all the database can be theoretically stored to a single tablespace.

⇒ Suggested that, separating different types of data into different tablespaces, can improve performance and queries.

Fault Tolerance:

⇒ if a data file in one tablespace becomes corrupted,
⇒ it is less likely to affect other tablespaces.

⇒ This can improve the overall reliability of the database.

TS	Description
SYSTEM	For data dictionary
DATA1...n	For data
INDEX1...m	For indexes
RBS	for standard rollback
TEMP	for temporary data
TOOLS1...n	for stored procedures & triggers

★ Allocating Data files:

- Parallel Processing

⇒ Technique that allows a database system to use multiple processors to execute query simultaneously.

Inter query Parallelism:

⇒ Technique that minimize conflicts between multiple queries that are running simultaneously.

⇒ To achieve this, different queries are stored on different disks. So, that queries can access the data independently.

2. Intra-query Parallelism:

⇒ Technique used to maximize the speed of individual queries.

⇒ To achieve this, the data required by a query is split into smaller chunks, and each chunk is processed by a separate processor.

Fault tolerance:

⇒ Ability of a system to continue operating even if one of its component fails.

⇒ This is important for DWH.

⇒ There are several techniques used to improve the fault tolerance of DWH:

- RAID
- Redundancy
- Backup & Recovery

(1) Redundancy:

⇒ Creating multiple copies of data and storing them on different disks.

⇒ If one disk fails, the data can be recovered from the other disk.

(2) RAID:

⇒ Redundant Array of independent disks

⇒ Technology that combines multiple disks into a single logical unit.

⇒ RAID can improve performance and fault tolerance.

⇒ Types of RAID configuration:

- RAID 1 (Mirroring)
- RAID 0+1 (Stripe+Mirror)
- RAID 5

Data Stripping:

- Refers to the technique used to allocate

a set of data to several disks to read and write at the same time.

→ To stripe data, split either single bits or entire data blocks.

Data block size:

- Number of bytes that are read or written to disk at one time.

Small disk block size:

- Can improve performance for random access queries

Large disk block size:

- Can improve performance for sequential access queries.